

Decision Tree Homework Solution

- **Problem:** Identify fraudulent transactions in a credit card transaction dataset
- **Dataset:** [Kaggle - Credit Card Fraud Detection \(https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud\)](https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud)
- **Classification (50 PTS):** DT, RF, LR, SVM — AUC, [F1@0.5 \(mailto:F1@0.5\)](mailto:F1@0.5), CPU time
- **Reflection (50 PTS):** Rankings, max F1, threshold discussion
- **Bonus (10 PTS):** Bootstrap OOB ~36% proof

```
In [2]: import time
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn import metrics
from sklearn.metrics import precision_recall_curve, f1_score
```

```
In [3]: # Download latest version
path = "D:\\dataset\\creditcard.csv"
print("Path to dataset files:", path)
```

Path to dataset files: D:\dataset\creditcard.csv

```
In [4]: df = pd.read_csv(path)
df = df.drop(columns=["Time", "Amount"])
print("shape of data = ", df.shape)
df[:2]
```

shape of data = (284807, 29)

Out[4]:

	V1	V2	V3	V4	V5	V6	V7	V8	V9
0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787
1	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425

2 rows × 29 columns



```
In [5]: df_label_1 = df[df['Class'] == 1]
df_label_0 = df[df['Class'] == 0]
df_label_0_sampled = df_label_0.sample(n=50000, random_state=123)
df_sampled = pd.concat([df_label_1, df_label_0_sampled]).reset_index(drop=True)
X_sampled = df_sampled.drop(columns=['Class']).values
labels_sampled = df_sampled.Class
print("Sampled data shape:", df_sampled.shape)
print(f"Positives: {labels_sampled.sum()} | Negatives: {(labels_sampled == 0).sum()}")
df_sampled.head(2)
```

Sampled data shape: (50492, 29)
Positives: 492 | Negatives: 50000

Out[5]:

	V1	V2	V3	V4	V5	V6	V7	V8	V9
0	-2.312227	1.951992	-1.609851	3.997906	-0.522188	-1.426545	-2.537387	1.391657	-2.770086
1	-3.043541	-3.157307	1.088463	2.288644	1.359805	-1.064823	0.325574	-0.067794	-0.270956

2 rows × 29 columns



```
In [6]: # Train/test split 70:30
X_train, X_test, y_train, y_test = train_test_split(
    X_sampled, labels_sampled, test_size=0.30, random_state=42
)
print(f"Train: {len(y_train)} | Test: {len(y_test)}")
```

Train: 35344 | Test: 15148

Classification (50 pts)

Train Decision Tree, Random Forest, Logistic Regression, SVM.
Report AUC, F1@threshold=0.5, and training time for each.

```

In [7]: results = {}

def train_evaluate(name, clf):
    t0 = time.time()
    clf.fit(X_train, y_train)
    elapsed = time.time() - t0

    proba = clf.predict_proba(X_test)[:, 1]
    pred = (proba >= 0.5).astype(int)

    fpr, tpr, _ = metrics.roc_curve(y_test, proba)
    auc = metrics.auc(fpr, tpr)
    f1 = f1_score(y_test, pred)

    results[name] = dict(proba=proba, auc=auc, f1=f1, time=elapsed, fpr=fpr, tpr=tpr)
    print(f" {name:25s}  AUC={auc:.4f}  F1@0.5={f1:.4f}  Time={elapsed:.2f}s")
    return proba

print(f"{'Model':25s}  {'AUC':>8}  {'F1@0.5':>8}  {'Time':>8}")
print("-" * 60)

p_dt = train_evaluate("Decision Tree",
                      DecisionTreeClassifier(class_weight="balanced", random_state=42))

p_rf = train_evaluate("Random Forest",
                      RandomForestClassifier(n_estimators=100, class_weight="balanced",
                                             random_state=42, n_jobs=-1))

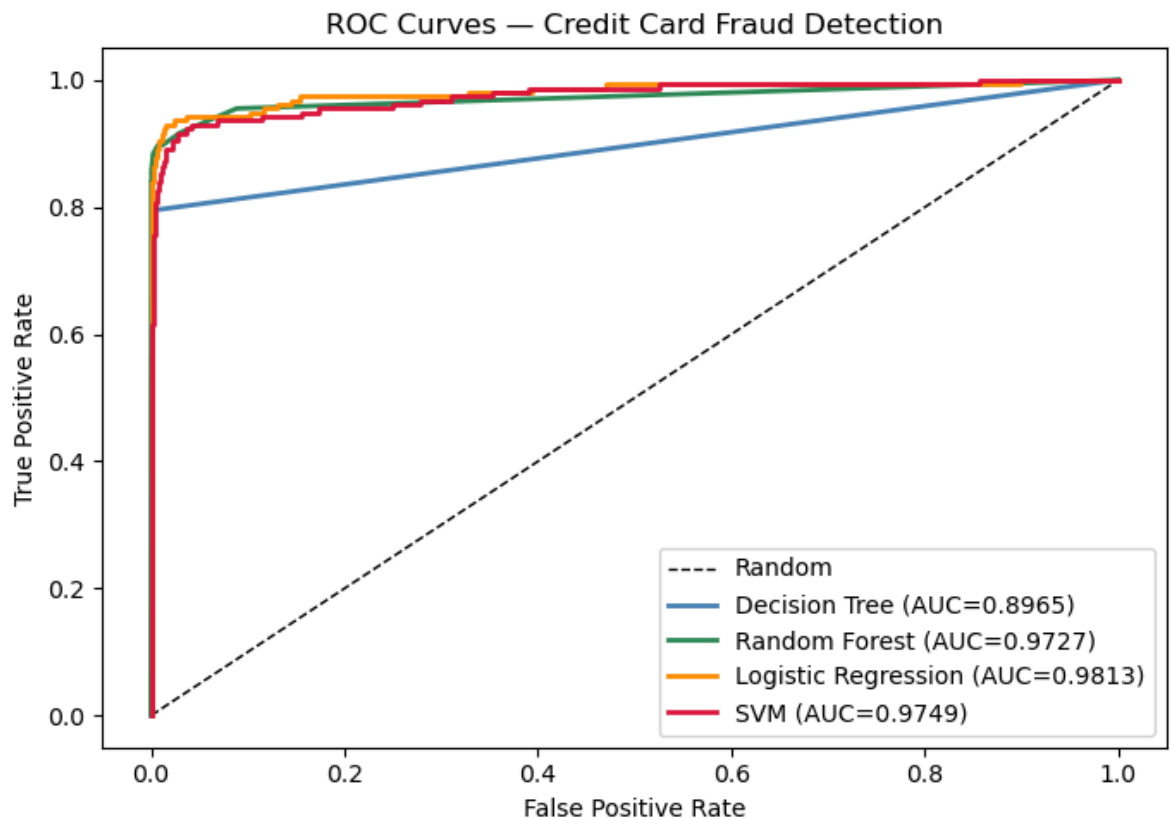
p_lr = train_evaluate("Logistic Regression",
                      LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42))

p_svm = train_evaluate("SVM",
                       SVC(kernel="rbf", class_weight="balanced", probability=True, random_state=

```

Model	AUC	F1@0.5	Time
Decision Tree	AUC=0.8965	F1@0.5=0.8026	Time=0.83s
Random Forest	AUC=0.9727	F1@0.5=0.8772	Time=0.88s
Logistic Regression	AUC=0.9813	F1@0.5=0.4567	Time=0.29s
SVM	AUC=0.9749	F1@0.5=0.6882	Time=43.38s

```
In [8]: # ROC Curve
fig, ax = plt.subplots(figsize=(7, 5))
ax.plot([0,1],[0,1], "k--", lw=1, label="Random")
for name, color in zip(["Decision Tree", "Random Forest", "Logistic Regression", "SVM"],
                        ["steelblue", "seagreen", "darkorange", "crimson"]):
    r = results[name]
    ax.plot(r["fpr"], r["tpr"], lw=2, color=color,
            label=f"{name} (AUC={r['auc']:.4f})")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("ROC Curves — Credit Card Fraud Detection")
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig("roc_curves_fraud.png", dpi=150)
plt.show()
```



Reflection (50 pts)

```
In [9]: # --- Ranking by AUC ---
print("RANKING BY AUC (best -> worst):")
for i, (n, r) in enumerate(sorted(results.items(), key=lambda x: x[1]['auc'], reverse=True)):
    print(f" {i}. {n}: {r['auc']:.4f}")

print("\nRANKING BY F1 @ threshold=0.5 (best -> worst):")
for i, (n, r) in enumerate(sorted(results.items(), key=lambda x: x[1]['f1'], reverse=True)):
    print(f" {i}. {n}: {r['f1']:.4f}")

print("\nRANKING BY CPU TIME (fastest -> slowest):")
for i, (n, r) in enumerate(sorted(results.items(), key=lambda x: x[1]['time']), 1):
    print(f" {i}. {n}: {r['time']:.2f}s")
```

RANKING BY AUC (best -> worst):

1. Logistic Regression: 0.9813
2. SVM: 0.9749
3. Random Forest: 0.9727
4. Decision Tree: 0.8965

RANKING BY F1 @ threshold=0.5 (best -> worst):

1. Random Forest: 0.8772
2. Decision Tree: 0.8026
3. SVM: 0.6882
4. Logistic Regression: 0.4567

RANKING BY CPU TIME (fastest -> slowest):

1. Logistic Regression: 0.29s
2. Decision Tree: 0.83s
3. Random Forest: 0.88s
4. SVM: 43.38s

```
In [10]: # Find max F1 per method
from sklearn.metrics import precision_recall_curve

def f1(p, r):
    denom = p + r
    denom = np.where(denom == 0, 1e-9, denom)
    return 2 * p * r / denom

def f1_max(proba):
    prec, recall, thresholds = precision_recall_curve(y_test, proba)
    f1_vals = f1(prec, recall)
    idx = np.argmax(f1_vals)
    best_thr = thresholds[idx] if idx < len(thresholds) else 1.0
    return f1_vals[idx], best_thr

print(f"{'Model':25s} {'Max F1':>8} {'Best Threshold':>16}")
print("-" * 55)
max_f1_results = {}
for name, proba in [("Decision Tree", p_dt), ("Random Forest", p_rf),
                    ("Logistic Regression", p_lr), ("SVM", p_svm)]:
    mfl, thr = f1_max(proba)
    max_f1_results[name] = mfl
    print(f"{'name':25s} {'mfl':>8.4f} {'thr':>16.4f}")
```

Model	Max F1	Best Threshold
Decision Tree	0.8026	1.0000
Random Forest	0.8973	0.2500
Logistic Regression	0.8789	0.9986
SVM	0.7316	0.2179

```
In [11]: print("RANKING BY MAX F1 (best -> worst):")
for i, (n, v) in enumerate(sorted(max_f1_results.items(), key=lambda x: x[1], reverse=True)):
    print(f"  {i}. {n}: {v:.4f}")
```

RANKING BY MAX F1 (best -> worst):

1. Random Forest: 0.8973
2. Logistic Regression: 0.8789
3. Decision Tree: 0.8026
4. SVM: 0.7316

Discussion: How to Compute F1

F1 at fixed threshold=0.5 is misleading for imbalanced datasets like fraud detection:

- The class imbalance means the model's predicted probabilities for positives are often well below 0.5, so a hard cutoff at 0.5 misses many true frauds.
- A model with high AUC can still show poor [F1@0.5 \(mailto:F1@0.5\)](#) if its probabilities are not calibrated around 0.5.

Max F1 (threshold-swept) is a fairer metric:

- By iterating over all possible thresholds using `precision_recall_curve`, we find the threshold that maximizes the F1 score for each model.
- This is analogous to AUC — it summarizes performance across thresholds rather than locking in an arbitrary one.

- The ranking by max F1 tends to agree much better with the AUC ranking than [F1@0.5](#) (<mailto:F1@0.5>) does.

Takeaway: When data is imbalanced, always tune the threshold or report max F1. Using [F1@0.5](#) (<mailto:F1@0.5>) alone can unfairly penalize models that are probabilistically well-separated but

Bonus (10 pts): Bootstrap OOB $\approx 36\%$

Show that when sampling n points with replacement from n points, $\sim 36\%$ of points are never selected.

Mathematical proof:

- $P(\text{a specific point is NOT selected in one draw}) = 1 - \frac{1}{n}$
- $P(\text{not selected in ALL } n \text{ draws}) = \left(1 - \frac{1}{n}\right)^n$
- As $n \rightarrow \infty$: $\left(1 - \frac{1}{n}\right)^n \rightarrow \frac{1}{e} \approx 0.3679$

So each bootstrap sample leaves out $\sim 36.8\%$ of the original points (out-of-bag samples).

```
In [12]: print(f"Theoretical limit: 1/e = {1/math.e:.6f} (~36.79%\n")

# Empirical simulation for increasing n
n_values = [100, 1000, 10000, 100000]
rng = np.random.default_rng(42)

print(f"{'n':>10} {'Empirical OOB':>15} {'(1-1/n)^n':>12}")
print("-" * 42)
for n in n_values:
    oob_fracs = []
    for _ in range(200):
        idx = rng.integers(0, n, size=n)
        oob_fracs.append(1 - len(np.unique(idx)) / n)
    empirical = np.mean(oob_fracs)
    theoretical = (1 - 1/n) ** n
    print(f"{n:>10} {empirical:>15.4f} {theoretical:>12.4f}")

print(f"\nConverges to 1/e = {1/math.e:.4f} as n -> infinity")
```

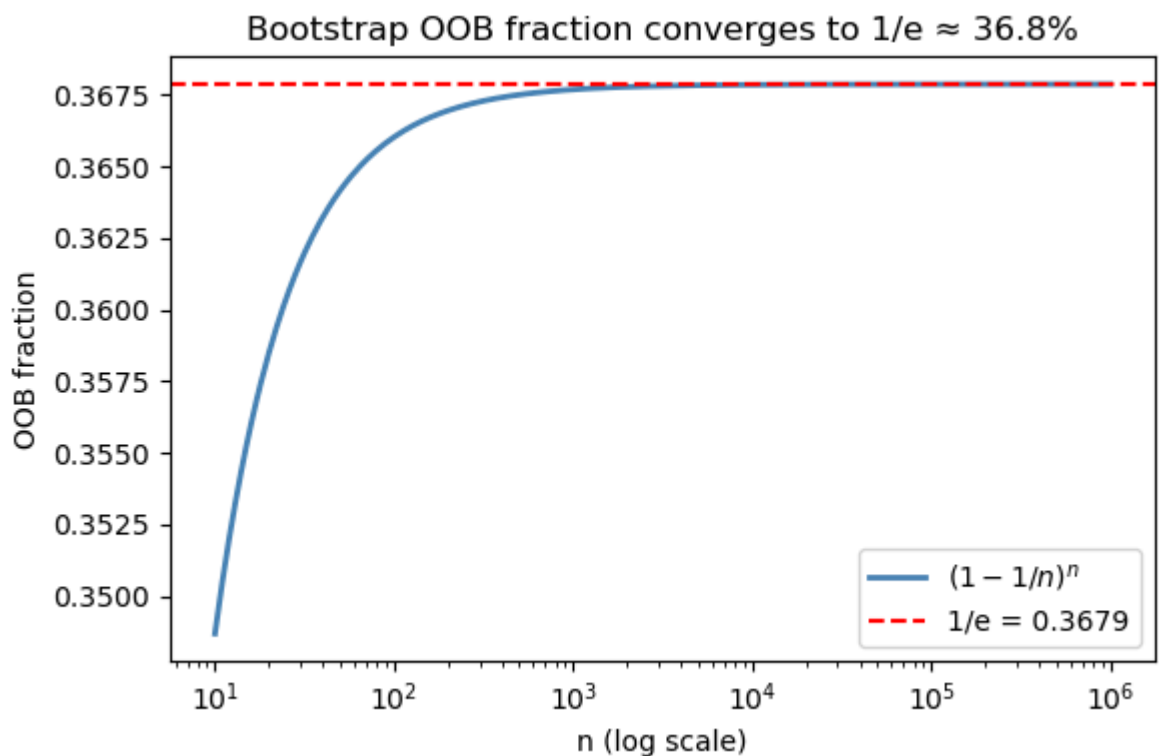
Theoretical limit: $1/e = 0.367879$ ($\sim 36.79\%$)

n	Empirical OOB	$(1-1/n)^n$
100	0.3686	0.3660
1000	0.3678	0.3677
10000	0.3679	0.3679
100000	0.3677	0.3679

Converges to $1/e = 0.3679$ as $n \rightarrow \infty$

```
In [13]: # Convergence plot
ns = np.logspace(1, 6, 300).astype(int)
curve = (1 - 1/ns) ** ns

fig, ax = plt.subplots(figsize=(6, 4))
ax.semilogx(ns, curve, lw=2, color="steelblue", label=r"$(1 - 1/n)^n$")
ax.axhline(1/math.e, color="red", linestyle="--", label=f"1/e = {1/math.e:.4f}")
ax.set_xlabel("n (log scale)")
ax.set_ylabel("OOB fraction")
ax.set_title("Bootstrap OOB fraction converges to 1/e ≈ 36.8%")
ax.legend()
plt.tight_layout()
plt.savefig("bootstrap_oob.png", dpi=150)
plt.show()
```



```
In [ ]:
```